

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ
ЖИВОТНОГО МИРА НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное учреждение
Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»
(ГБПОУ НО «КБЛК»)

ДЕНЬ 12

**Отладка программы с использованием специализированных средств
отладки**

Выполнила: Прямых Надежда,

Студентка 24-ИС.

Руководитель: Торопов Е.В.

р.п. Красные Баки

2026 г.

Вариант 4

- Классы с slots
- AttributeError (неверное имя атрибута)
- Неправильное обновление баланса
- Ссылки в dict (если без slots)
- Переопределение getattr

```
import tracemalloc
CACHE = []
class Account:
    __slots__ = ['id', 'balance', 'owner', 'transactions']

    def __init__(self, account_id, owner, initial_balance=0):
        self.id = account_id
        self.owner = owner
        self.balance = initial_balance
        self.transactions = []

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Сумма должна быть положительной")
        self.balance += amount
        self.transactions.append(f"+{amount}")
        return self.balance

    def withdraw(self, amount):
        if amount <= 0:
            raise ValueError("Сумма должна быть положительной")
        self.balance -= amount
        self.transactions.append(f"-{amount}")
        return self.balance

    def __getattr__(self, name):
        print(f"Атрибут '{name}' не найден, возвращаем 0")
        return 0

class Bank:
    def __init__(self):
        self.accounts = {}

    def create_account(self, owner, initial_balance=0):
        account_id = len(self.accounts) + 1
        account = Account(account_id, owner, initial_balance)
        self.accounts[account_id] = account
        CACHE.append(account)
        return account

    def transfer(self, from_id, to_id, amount):
        if from_id not in self.accounts:
```

```

        raise KeyError(f"Счет {from_id} не найден")
    if to_id not in self.accounts:
        raise KeyError(f"Счет {to_id} не найден")

    from_account = self.accounts[from_id]
    to_account = self.accounts[to_id]

    from_account.withdraw(amount)
    to_account.deposit(amount)
    return True

def recursive_operation(account, depth):
    if depth <= 0:
        return account.balance
    return recursive_operation(account, depth - 1)

def main():
    tracemalloc.start()

    bank = Bank()

    acc1 = bank.create_account("Иванов", 1000)
    acc2 = bank.create_account("Петров", 500)
    acc3 = bank.create_account("Сидоров", 200)

    bank.transfer(1, 2, 300)
    bank.transfer(2, 3, 100)

    print(acc1.non_existent_attribute)

    recursive_operation(acc1, 5)

    snapshot = tracemalloc.take_snapshot()
    print("\nТоп-5 строк по памяти:")
    for stat in snapshot.statistics('lineno')[:5]:
        print(stat)
    print(f"\nCACHЕ содержит {len(CACHЕ)} объектов")

if __name__ == "__main__":
    main()

```

C:\Users\Student234\AppData\Local\Python\pythoncore-3.14-64\Lib\idlelib\rpc.py:2
30: size=216 В, count=1, average=216 В
Y:\24 ИСиП 2025\УП02\Группа 2\Прямых Н\д 12\variant_4.py:38: size=192 В, count=3
, average=64 В

CACHЕ содержит 3 объектов

Запуск программы с ошибками:

```

> variant_4.py(75)main()
-> bank = Bank()
(Pdb) n
> variant_4.py(78)main()
-> acc1 = bank.create_account("Иванов", 1000)
(Pdb) s
--Call--
> variant_4.py(38)create_account()
-> def create_account(self, owner, initial_balance=0):
(Pdb) n
> variant_4.py(39)create_account()
-> account_id = len(self.accounts) + 1
(Pdb) n
> variant_4.py(40)create_account()
-> account = Account(account_id, owner, initial_balance)
(Pdb) n
> variant_4.py(41)create_account()
-> self.accounts[account_id] = account
(Pdb) p account
Account(id=1, owner=Иванов, balance=1000)
(Pdb) n
> variant_4.py(42)create_account()
-> CACHE.append(account)
(Pdb) n
> variant_4.py(43)create_account()
-> return account
(Pdb) n
--Return--
> variant_4.py(43)create_account()-><__main__.Acco...0x000001C8A1F>
-> return account
(Pdb) n
> variant_4.py(81)main()
-> acc2 = bank.create_account("Петров", 500)

```

Работа в pdb:

```
import tracemalloc
```

```
import weakref
```

```
CACHE = weakref.WeakSet()
```

```
class Account:
```

```
    __slots__ = ['id', 'balance', 'owner', 'transactions', '__weakref__']
```

```
    def __init__(self, account_id, owner, initial_balance=0):
```

```
        self.id = account_id
```

```
        self.owner = owner
```

```
        self.balance = initial_balance
```

```
        self.transactions = []
```

```
    def deposit(self, amount):
```

```
        if amount <= 0:
```

```
            raise ValueError("Сумма должна быть положительной")
```

```
self.balance += amount
self.transactions.append(f"+{amount}")
return self.balance
```

```
def withdraw(self, amount):
    if amount <= 0:
        raise ValueError("Сумма должна быть положительной")
    if self.balance < amount:
        raise ValueError("Недостаточно средств")
    self.balance -= amount
    self.transactions.append(f"-{amount}")
    return self.balance
```

```
def __getattr__(self, name):
    raise AttributeError(f"'{self.__class__.__name__}' has no attribute '{name}'")
```

```
def __repr__(self):
    return f"Account(id={self.id}, owner={self.owner}, balance={self.balance})"
```

```
class Bank:
```

```
    def __init__(self):
        self.accounts = {}
```

```
    def create_account(self, owner, initial_balance=0):
        account_id = len(self.accounts) + 1
        account = Account(account_id, owner, initial_balance)
        self.accounts[account_id] = account
        CACHE.add(account)
        return account
```

```
    def transfer(self, from_id, to_id, amount):
        if from_id not in self.accounts:
            raise KeyError(f"Счет {from_id} не найден")
        if to_id not in self.accounts:
```

```
raise KeyError(f"Счет {to_id} не найден")
```

```
from_account = self.accounts[from_id]
```

```
to_account = self.accounts[to_id]
```

```
from_account.withdraw(amount)
```

```
to_account.deposit(amount)
```

```
return True
```

```
def recursive_operation(account, depth):
```

```
    if depth == 0:
```

```
        return account.balance
```

```
    return recursive_operation(account, depth - 1)
```

```
def main():
```

```
    tracemalloc.start()
```

```
    print("=" * 50)
```

```
    print("ЗАПУСК (ИСПРАВЛЕННАЯ ВЕРСИЯ)")
```

```
    print("=" * 50)
```

```
    bank = Bank()
```

```
    acc1 = bank.create_account("Иванов", 1000)
```

```
    acc2 = bank.create_account("Петров", 500)
```

```
    acc3 = bank.create_account("Сидоров", 200)
```

```
    print("\nБалансы до переводов:")
```

```
    print(f"  Иванов: {acc1.balance}")
```

```
    print(f"  Петров: {acc2.balance}")
```

```
    print(f"  Сидоров: {acc3.balance}")
```

```
    bank.transfer(1, 2, 300)
```

```

bank.transfer(2, 3, 100)

print("\nБалансы после переводов:")
print(f" Иванов: {acc1.balance}")
print(f" Петров: {acc2.balance}")
print(f" Сидоров: {acc3.balance}")

print("\nПроверка __getattr__:")
try:
    print(acc1.non_existent)
except AttributeError as e:
    print(f" Правильно сгенерировано: {e}")

print("\nПроверка рекурсии:")
result = recursive_operation(acc1, 3)
print(f" Результат: {result}")

snapshot = tracemalloc.take_snapshot()
print("\nТоп-5 строк по памяти:")
for stat in snapshot.statistics('lineno')[:5]:
    print(f" {stat}")
print(f"\nCACHE содержит {len(CACHE)} объектов")
print("\n" + "=" * 50)
print("ВСЕ ТЕСТЫ ПРОЙДЕНЫ УСПЕШНО!")

if __name__ == "__main__":
    main()

Y:\24 ИСиП 2025\УП02\Группа 2\Прямых Н\д 12\variant_4_fixed.py:97: size=280 B,
count=1, average=280 B
C:\Users\Student234\AppData\Local\Programs\Python\Python314\Lib\idlelib\rpc.py
:342: size=240 B, count=4, average=60 B

CACHE содержит 3 объектов

=====
ВСЕ ТЕСТЫ ПРОЙДЕНЫ УСПЕШНО!

```

Успешный запуск исправленной программы:

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

A1. Ответ: Стек вызовов - это структура данных, хранящая последовательность вызовов функций. В момент ошибки его можно посмотреть через `traceback.print_exc()` или командой `where` в `pdb`.

A2. Ответ: `p` var выводит значение в формате `repr()`, а `pp` var использует `pprint()` для красивого форматирования сложных структур.

A3. Ответ: Командой `up`. Для возврата обратно используется `down`.

A4. Ответ: `tracemalloc` отслеживает выделение памяти. Для топ-10: `snapshot = tracemalloc.take_snapshot(); top_stats = snapshot.statistics('lineno')[:10]`

A5. Ответ: `python -m pdb script.py`

A6. Ответ: потому что перехватывает все исключения включая системные, маскируя ошибки и затрудняя отладку.

A7. `tracemalloc` - встроенный модуль для трассировки выделений памяти.

A8. `import traceback; traceback.print_exc()` или использовать `logging.exception()`

A9. `RecursionError` возникает при превышении максимальной глубины рекурсии (по умолчанию 1000). Избегать можно: установкой `sys.setrecursionlimit()`, преобразованием рекурсии в итерацию, добавлением корректного базового случая.

A10. Для перемещения на один уровень вверх по стеку вызовов (к родительской функции), чтобы просмотреть контекст вызвавшей функции.

B1. `breakpoint()` появился в Python 3.7 и является более удобным: он использует настройки из переменной окружения `PYTHONBREAKPOINT` и может быть отключен глобально. `pdb.set_trace()` жестко привязан к `pdb`.

B2. Использовать условный брейкпоинт:
break filename:line, condition var == value
condition 1 var == value

B3. Засоряет код и вывод программы, требует удаления после отладки, не дает интерактивности, не работает в продакшене без перезапуска, не имеет уровней логирования, может замедлять выполнение

B4. Запустить tracemalloc.start() при старте приложения, периодически снимать снимки: snapshot1 = tracemalloc.take_snapshot(), после каждого запроса/цикла снимать новый снимок и сравнивать: snapshot2.compare_to(snapshot1, 'lineno'), отслеживать рост памяти по строкам кода, настроить автоматический отчет при превышении порога памяти

Вывод:

В ходе выполнения практической работы по отладке программы на языке Python были изучены и применены основные методы и инструменты отладки. На примере Варианта 4 были выявлены и исправлены следующие типы ошибок: логическая ошибка в методе withdraw , ошибка в переопределении __getattr__, ошибка рекурсии, а также утечка памяти из-за бесконечного роста глобального кеша.

В процессе работы были освоены интерактивный отладчик pdb, модуль tracemalloc для отслеживания утечек памяти, а также правильная обработка исключений. Все ошибки успешно исправлены, программа работает корректно, утечек памяти не обнаружено.